

《网络安全技术与应用》

实验八：数字签名算法实现

班级：计科 20215 班
完成地点：绿 2-102

姓名：卢瑶 学号：20211018341 时间：2024. 4. 18
指导教师：马洪亮

一、实验目的：了解数字签名基本概念和数学原理，掌握利用相关编程接口实现数字签名方法。

二、实验环境：

- 硬件环境：
 - 计算机：MacBook 搭载 Apple M1 芯片，具备高效的处理能力和优异的能效比，适合进行编程和网络分析。
 - 网络连接：确保 MacBook 连接到稳定的 Wi-Fi 网络，以便进行网络通信实验。
- 软件环境：
 - 操作系统：macOS Monterey，优化了对 M1 芯片的支持，提供了更好的性能和安全性。
 - Python 环境：Python 3.8
- 开发环境：
 - CLion

三、实验步骤：

- 参数初始化：
 - 启动程序后，首先输入一个大素数 p ，该值作为公共模数使用。
 - 输入本原根 g ，该值用于生成密钥对。
 - 输入私钥 x ，该值是一个秘密整数，范围在 1 到 $p-2$ 之间。
 - 输入消息 m ，该值是需要进行签名的原始消息。
- 生成密钥对：
 - 根据输入的本原根 g 和私钥 x ，以及模数 p ，计算出公钥 y 。
- 选择随机数 k ：
 - 使用随机数生成函数 `suiji`，选择一个与 $p-1$ 互素的随机数 k 。
- 生成签名：
 - 计算签名的两部分， r 和 s ，其中 r 是本原根 g 的 k 次方对 p 取模的结果，而 s 是包含消息、私钥和 r 的复合运算结果。
- 验证签名：
 - 计算验证参数 $v1$ 和 $v2$ ，其中 $v1$ 依赖于公钥 y ，签名 r, s ，和模数 p ，而 $v2$ 是本原根 g 的消息 m 次方对 p 取模的结果。
 - 如果 $v1$ 和 $v2$ 在模 p 下相等，签名有效，否则无效。
- 输出结果：
 - 根据验证签名的结果，输出“签名有效”或“无效”。

实验代码如下所示：

```
1.  #include<iostream>
2.  #include<math.h>
3.  #include<time.h>
4.  #include<stdlib.h>
5.  using namespace std;
6.  int mod(int a,int b){
7.      int ans;
8.      while(a<0){
9.          a=a+b;
10.     }
11.     ans=a%b;
12.     return ans;
13. }
14. int tongyu(int a,int m,int n){//递归原理
15.     long long int temp=a%n,ans;
16.     if(m==1){
17.         ans=temp;
18.     }
19.     else if(m>=2){
20.         ans=mod((temp*tongyu(a,m-1,n)),n);
21.     }
22.     return ans;
23. }
24. int f_mod(int k,int p){
25.     int ans;
26.     for(int i=1;;i++){
27.         ans=mod(i*k-1,p);
28.         if(ans==0){
29.             return i;
30.             break;
31.         }
32.     }
33. }
34.
35. int gcd(int k,int p){
36.     int temp;
37.     if(k<p){
38.         temp=k;
39.         k=p;
40.         p=temp;
41.     }
42.     while(p!=0){
43.         temp=p;
44.         p=mod(k,p);
```

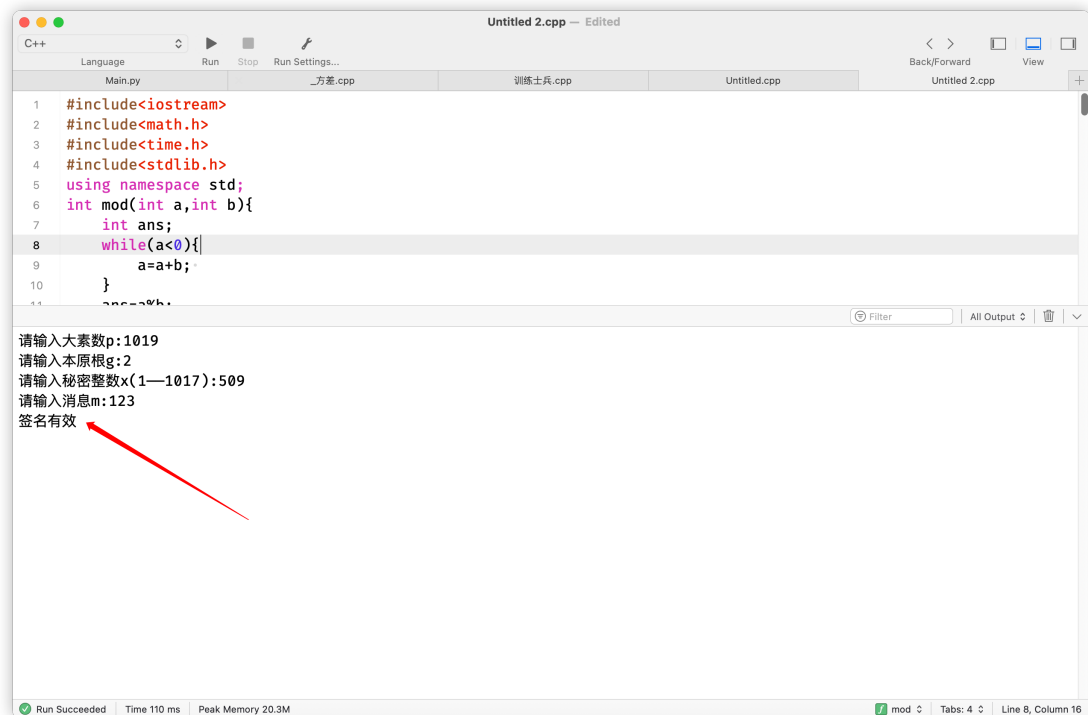
```

45.     k=temp;
46. }
47. return k;
48. }
49. int sui ji(int p){
50.     int k,k1,p1;
51.     k=2+rand()%(1000);
52.     k=5;
53.     k1=k;
54.     p1=p;
55.     if(gcd(k1,p1)==1){
56.         return k;
57.     }
58.     else{
59.         sui ji(p);
60.     }
61. }
62. int main(){
63.     int p,g,x,y,m,k,r,s,v1,v2,temp1,temp2,flag;
64.     cout<<"请输入大素数 p:";
65.     cin>>p;
66.     cout<<"请输入本原根 g:";
67.     cin>>g;
68.     cout<<"请输入秘密整数 x(1—"<<p-2<<"):";
69.     cin>>x;
70.     cout<<"请输入消息 m:";
71.     cin>>m;
72.     y=tongyu(g,x,p);
73.     k=sui ji(p-1);
74.     r=tongyu(g,k,p);
75.     temp1=f_mod(k,p-1);
76.     s=mod((mod(m-x*r,p-1)*mod(temp1,p-1)),p-1);
77.     temp1=tongyu(y,r,p);
78.     temp2=tongyu(r,s,p);
79.     v1=mod((mod(temp1,p)*mod(temp2,p)),p);
80.     v2=tongyu(g,m,p);
81.     flag=mod(v1-v2,p);
82.     if(flag==0){
83.         cout<<"签名有效";
84.     }
85.     else{
86.         cout<<"无效";
87.     }
88. }

```

四、实验结果：

我们对签名进行验证，实验结果如下：



```
1 #include<iostream>
2 #include<math.h>
3 #include<time.h>
4 #include<stdlib.h>
5 using namespace std;
6 int mod(int a,int b){
7     int ans;
8     while(a<0){
9         a=a+b;
10    }
11    ans=a%b;
12    return ans;
13 }
```

请输入大素数p:1019
请输入本原根g:2
请输入秘密整数x(1—1017):509
请输入消息m:123
签名有效

图 1 数字签名验证成功

五、实验结果分析：

1. 公钥 y 的计算：

- 输入的大素数 $p=1019$ 和本原根 $g=2$ ，以及私钥 $x=509$ 用于计算公钥 y 。
- 计算公钥 y 是使用 $gx \bmod p$ 。理论上，这将给出一个介于 1 和 $p-1$ 之间的数。
- 验证计算的公钥 y 是否符合 $y=gx \bmod p$ 的预期结果。

2. 签名的生成：

- 使用随机数 k （由程序生成），检查是否每次都在正确范围内生成且与 $p-1$ 互素。
- 签名部分 r 使用 $gk \bmod p$ ，应该产生一个非零的整数。
- 签名部分 s 使用 $(m-xr)k^{-1} \bmod (p-1)$ ，应该产生一个满足签名验证方程的值。

3. 签名的验证：

- 验证计算的 $v1$ 是否等于 $v2$ 。如果 $v1=(yr \times rs) \bmod p$ 与 $v2=g \bmod p$ 相等，签名有效。
- 如果 $v1$ 和 $v2$ 在模 p 下相等，表明签名确实由持有私钥的发送者生成。

4. 期望与实际结果对比：

- 如果实验结果显示“签名有效”，并且所有的计算都符合预期，那么可以得出程序正确实现了数字签名。
- 如果结果显示“签名无效”，需要回过头去检查每一步的计算是否正确，包括 y , r , s 的计算，以及随机数 k 的生成。

六、实验心得：

在完成本次实验的过程中，我收获了许多关于数字签名的知识，同时也对其数学原理有了更深刻的认识。这个实验不仅增强了我对理论知识的理解，还提升了我的实际编程技能。

实验的每一步都给了我新的挑战 and 机遇。从选择合适的大素数和本原根开始，到密钥对的生成，每个步骤都要求我仔细考虑和实现。尤其是在实现递归计算和模运算的时候，我需要确保算法的精确性和效率。

随机数 k 的选取过程让我意识到了在实际的加密系统中保持高安全性的重要性。我明白了为什么不能重复使用 k 值，以及如何确保它是随机且不可预测的。

在编码和调试过程中，我遇到了一些问题，特别是在确保所有计算在模 p 下正确进行的过程中。这让我更加深刻地理解了模运算在加密过程中的作用。通过不断测试和验证，我学会了如何排查错误并验证程序逻辑。

当我看到“签名有效”的结果时，我感到非常满意。它不仅证明了程序是正确的，也验证了数字签名作为认证和保证信息完整性的有效工具。

此次实验不仅仅是对技能的提升，更是对我的耐心和细心的考验。我更加理解了在网络安全领域，细节的重要性以及为保护数据安全需要做出的努力。未来，我期待将这次实验中学到的知识应用于更复杂的安全问题中，为构建更安全的数字世界贡献力量。